

PO Long Text New Persistency — Skill Reference Guide v2.0

PO Long Text New Persistency — Skill Reference Guide

Project: PO Long Text Migration to New Persistence Table **Skill Name:** po-longtext-persistency-expert **Location:** C:\Users\I308878\PO_LongText_Persistency **Date:** 2026-06-19 **Version:** 2.0 — corrected to single-system architecture

1. The Concept — How SAP Stores Long Texts

SAP uses a **generic text framework** to store free-form text across all business objects (POs, SOs, Material Masters, Vendors etc.). Two tables are involved:

STXH — Text Header

One row per text. Stores only metadata — no content.

Field	Description	Example
TDOBJECT	Which object type	EKKO (PO Header) / EKPO (PO Item)
TDID	Which type of note	F01 (General) / F02 (Delivery)
TDSPRAS	Language	E (English) / D (German)
TDNAME	Document key	4500001234 (PO number)

STXL — Text Lines (COMPRESSED)

Stores the actual text content in **binary compressed cluster format**.

Critical: STXL cannot be read with SE16 or plain SELECT. You must use the function module READ_TEXT which decompresses on the fly.

```
" The ONLY way to read SAP long text content:
CALL FUNCTION 'READ_TEXT'
  EXPORTING
    id          = 'F01'          " Text ID
```

```
language = 'E'           " Language
name      = '4500001234'  " TDNAME (PO number for header)
object    = 'EKKO'        " Text object
TABLES
  lines    = lt_lines.     " Returns flat TLINE table (TDFORMAT + TDLINE)
```

Why This Is a Problem

Problem	Impact
STXL not SQL-readable	Cannot use in CDS views or reports
READ_TEXT required for every read	Performance degrades at scale
BDC programs	Must call READ_TEXT FM — complex and slow
Decompression overhead	Expensive per call for millions of POs

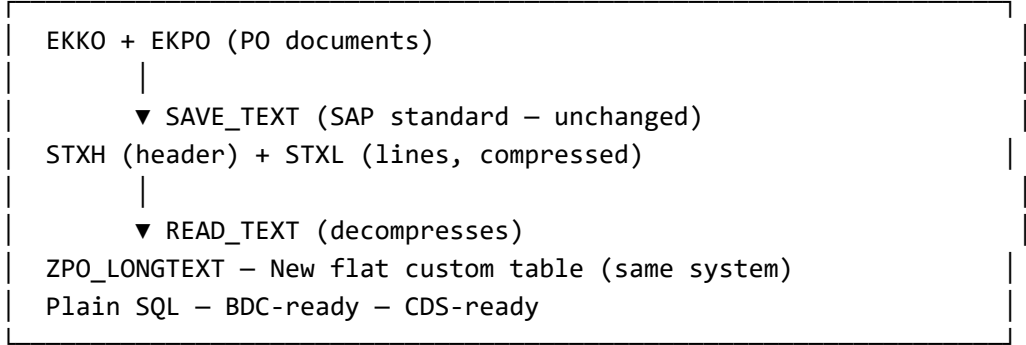
2. The Solution — New Flat Persistence Table

Create a custom table ZPO_LONGTEXT on the **same SAP system** that stores the decompressed, flat text lines. This table is:

- Plain SQL-readable — no decompression needed
- Usable directly in CDS views, reports, BDC programs
- Populated by a one-time migration program AND by a BAdI on every PO save
- **Parallel to STXH/STXL — does NOT replace them**

System Landscape — Single System

SINGLE SAP SYSTEM (ERP / S/4HANA)



No Central Hub. No replication to another system. Everything on one SAP system.

3. New Persistence Table: ZPO_LONGTEXT

Field	Key	Type	Length	Description
MANDT	✓	CLNT	3	Client
EBELN	✓	CHAR	10	Purchase Order Number
EBELP	✓	NUMC	5	PO Item (00000 = header)
TDOBJECT	✓	CHAR	10	EKKO or EKPO
TDID	✓	CHAR	4	F01 / F02 / F09 etc.
TDSPRAS	✓	LANG	1	Language Key
LINE_COUNTER	✓	NUMC	5	Sequence (00001, 00002...)
TDFORMAT		CHAR	2	Format indicator
TDLINE		CHAR	132	Text line content — plain text
MIGRATED_AT		DATS	8	Date written
SOURCE		CHAR	1	M=Migration, D=Dual-write

4. Text Types Reference

TDOBJECT	TDID	Level	Description	EBELP in ZPO_LONGTEXT
EKKO	F01	Header	PO Header General Note	00000
EKKO	F02	Header	PO Header Delivery Text	00000
EKKO	F03	Header	PO Header Terms of Payment	00000
EKPO	F01	Item	PO Item General Note	item number
EKPO	F02	Item	PO Item Delivery Text	item number
EKPO	F09	Item	PO Item GR Text	item number

TDNAME Construction in STXH

Level	TDNAME Format	Example
Header (EKKO)	EBELN (10 chars)	4500001234
Item (EKPO)	EBELN + space + EBELP	4500001234 00010

Getting TDNAME wrong makes READ_TEXT return nothing. Always verify in SE16 → STXH.

5. Skill Structure

```

PO_LongText_Persistenceency/
├── Claude.md                    ← Skill definition + full ABAP
code
├── Trigger words, architecture,
│   ├── ZCL_PO_LONGTEXT_HANDLER,
│   ├── ZCL_PO_TEXT_BADI_IMPL,
│   └── ZPO_LONGTEXT_MIGRATE skeleton
├── repos.yaml                  ← Source document registry
└── agent/
    ├── knowledge/
    │   ├── architecture/
    │   │   ├── po-text-flow-diagram.md    ← Architecture diagrams
    │   │   │   ├── Single system layout
    │   │   │   ├── Phase 1 migration flow
    │   │   │   ├── Phase 2 dual-write flow
    │   │   │   ├── TDNAME construction
    │   │   │   └── Debugging decision tree
    │   │   └── domain/
    │   │       ├── po-longtext-domain.md    ← Domain concepts
    │   │       │   ├── Classic framework explained
    │   │       │   ├── Text types reference
    │   │       │   ├── Migration rules
    │   │       │   └── SO parallel pattern
    │   └── codebase-index/
    │       ├── codebase-overview.md        ← Object index
    │       ├── business-rules.md          ← 7 invariant rules
    │       └── data-flows.md              ← 3 flows: Migrate, Dual-write,
BDC Read

```

6. Architecture — Two Phases

Phase 1 — Historic Migration (One-Time)

```

ZPO_LONGTEXT_MIGRATE (SE38 report)
↓
SELECT STXH WHERE TDOBJECT IN ('EKKO','EKPO')
↓
For each row: parse TDNAME → EBELN + EBELP
↓
CALL FUNCTION 'READ_TEXT' → decompress → flat lines
↓
DELETE old ZPO_LONGTEXT rows for this text
INSERT new flat lines (SOURCE = 'M')
COMMIT WORK every 1000 POs

```

↓
Validate: count STXH rows vs ZPO_LONGTEXT rows

Phase 2 — Dual Write (Ongoing)

User saves PO in ME21N / ME22N
↓
SAP standard: SAVE_TEXT → STXH + STXL (unchanged)
↓
BAdI: ME_PROCESS_PO_CUST fires
 Method PROCESS_HEADER → header texts
 Method PROCESS_ITEM → item texts
↓
ZCL_PO_TEXT_BADI_IMPL
↓
ZCL_PO_LONGTEXT_HANDLER→write_to_persistence()
↓
READ_TEXT → DELETE old → INSERT new (SOURCE = 'D')
↓
STXH/STXL and ZPO_LONGTEXT always in sync

BDC / Reporting Read

Before (slow): CALL FUNCTION 'READ_TEXT' → decompress STXL
After (fast): SELECT * FROM zpo_longtext WHERE ebeln = '<PO>'
 ORDER BY tdoobject, tddid, tdspras, line_counter.

7. Key Business Rules

#	Rule	Why
1	ZPO_LONGTEXT is parallel — NEVER replace STXH/STXL	SAP standard depends on them
2	EBELP = ‘00000’ for TDOBJECT = ‘EKKO’ header texts	EBELP is a key field — NULL breaks queries
3	TDNAME for EKPO = EBELN(10) + space + EBELP — parse carefully	Wrong TDNAME → READ_TEXT returns nothing
4	ORDER BY LINE_COUNTER ASCENDING always	Text lines must render in correct sequence
5	DELETE + INSERT per text (not UPSERT)	Deleted lines on STXH must be removed from ZPO_LONGTEXT
6	BAdI fires AFTER SAVE_TEXT —	Calling READ_TEXT before SAVE_TEXT

#	Rule	Why
7	READ_TEXT in BAdI reads fresh text	returns nothing
	Run migration in TEST mode first — verify counts before productive	Prevents partial writes on errors

8. Key ABAP Objects

Object	Type	Description
ZPO_LONGTEXT	Table	New flat persistence table
ZPO_LONGTEXT_MIGRATE	Report (SE38)	One-time migration from STXH/STXL
ZCL_PO_LONGTEXT_HANDLER	Class	Central handler — READ_TEXT wrapper + ZPO_LONGTEXT writer
ZCL_PO_TEXT_BADI_IMPL	Class	BAdI impl — fires on ME21N/ME22N save
ME_PROCESS_PO_CUST	BAdI	Enhancement Spot ME_PURCHORD — PO save hook
STXH	SAP table	Text header (source — do not modify)
STXL	SAP table	Text lines compressed (source — do not modify)
READ_TEXT	Function Module	Decompresses STXL → flat lines
SAVE_TEXT	Function Module	Writes STXH + STXL (SAP standard)

9. Debugging Guide

"PO 4500001234 header note not in ZPO_LONGTEXT"

Step 1: Check STXH — does text exist on ERP?

SE16 → STXH WHERE TDOBJECT='EKKO' AND TDNAME='4500001234' AND TDID='F01'

If NO → text never created. Not a persistence issue.

Step 2: Check ZPO_LONGTEXT

SE16 → ZPO_LONGTEXT WHERE EBELN='4500001234' AND TDOBJECT='EKKO' AND TDID='F01'

If NO → run ZPO_LONGTEXT_MIGRATE for this PO, or check BAdI is active

Step 3: Check BAdI active

SE19 → ME_PROCESS_PO_CUST → find ZCL_PO_TEXT_BADI_IMPL

Must show status = Active

Step 4: Check LINE_COUNTER sequence

SELECT from ZPO_LONGTEXT ORDER BY line_counter

Gaps → partial migration failure → re-run for this PO

10. What Makes It Extensible

Adding New Text Types (e.g. TDID = 'F03')

1. Add iv_tdid = 'F03' call in ZCL_PO_TEXT_BADI_IMPL→PROCESS_HEADER
2. No table change needed — TDID is a key field in ZPO_LONGTEXT
3. Re-run migration for existing POs that have F03 text

Sales Order Parallel (VBAK/VBAP)

Same architecture — create ZSO_LONGTEXT with same structure. Change TDOBJECT values: EKKO/EKPO → VBAK/VBAP. BAdI: use user-exit in VA01/VA02 instead of ME_PROCESS_PO_CUST.

Adding New Knowledge Documents

Place new handover documents in: - agent/knowledge/domain/ — domain rule updates - agent/knowledge/product/ — SDD and design decisions - Update repos.yaml source_documents list

*Skill created: 2026-06-19 | Version 2.0 — corrected to single-system architecture Source: PO Notes New Persistency handover documents *Skill file: C:_LongText_Persistency.md**